

Cartes de Kohonen

Self-Organizing Maps (SOM)

STA211 — Apprentissage non supervisé — Fiche récapitulative
8 juin 2026

Table des matières

1	Présentation	2
2	Architecture	2
3	Algorithme	2
3.1	Principe général	2
3.2	Notations	3
3.3	Étape 1 — Initialisation	3
3.4	Étape 2 — Recherche du BMU (Best Matching Unit)	3
3.5	Étape 3 — Mise à jour des poids	3
3.6	Fonction de voisinage	3
3.7	Plan de décroissance	3
3.8	Pseudo-code	3
4	Exemple numérique pas à pas	4
4.1	Données	4
4.2	Architecture	4
4.3	Initialisation	4
4.4	Itération 1 — Individu $\mathbf{x}^{(1)} = (0,2; 0,3)$	4
4.5	Itération 1 — Individu $\mathbf{x}^{(2)} = (0,1; 0,8)$	5
4.6	Itération 1 — Individu $\mathbf{x}^{(3)} = (0,7; 0,2)$	5
4.7	Itération 1 — Individu $\mathbf{x}^{(4)} = (0,9; 0,7)$	5
4.8	Interprétation	5
5	Code Python	6
5.1	Implémentation simple avec NumPy	6
5.2	Application sur le jeu de données Iris	6
5.3	Avec la bibliothèque MiniSom	7
6	Code R	7
7	Diagnostic et visualisations	8
7.1	U-Matrix (Unified Distance Matrix)	8
7.2	Carte des effectifs (<i>counts</i>)	8
7.3	Plans de composantes (<i>component planes</i>)	8
7.4	Carte de qualité	8
8	Résumé — Points clés	9

1 Présentation

Les **cartes de Kohonen** (ou *Self-Organizing Maps*, SOM) sont un type de réseau de neurones artificiels utilisé en **apprentissage non supervisé**. Inventées par Teuvo Kohonen dans les années 1980, elles permettent de **visualiser des données de grande dimension** en les projetant sur une grille (souvent 2D) tout en préservant les relations de voisinage.

Objectifs :

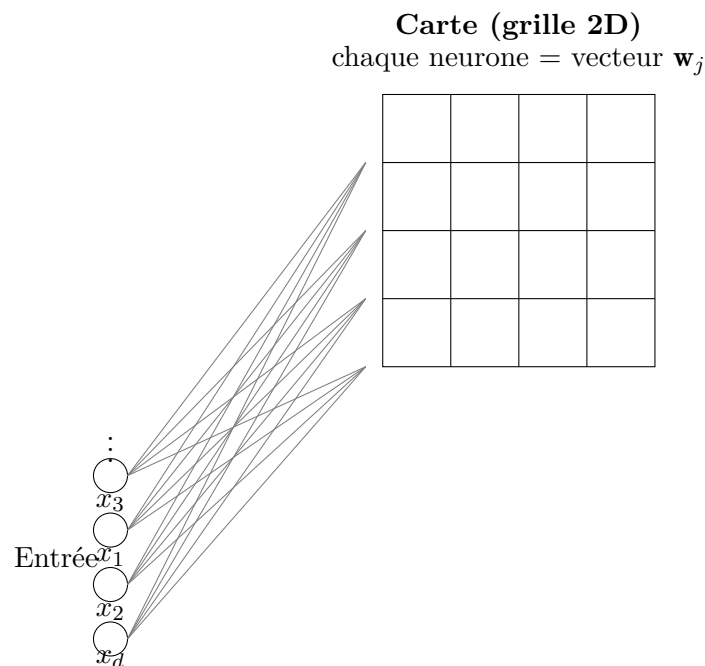
- Réduction de dimension (visualisation)
- Clustering (segmentation non supervisée)
- Conservation de la topologie des données

Propriété clé : Deux points proches dans l'espace d'entrée sont représentés par des neurones proches (ou identiques) sur la carte.

2 Architecture

La carte de Kohonen est composée de deux couches :

- **Couche d'entrée** : chaque neurone d'entrée correspond à une variable (une dimension des données).
- **Couche de sortie (carte)** : une grille de neurones organisés spatialement (souvent rectangulaire ou hexagonale). Chaque neurone de sortie j possède un **vecteur de poids $\mathbf{w}_j$** de même dimension que les données d'entrée.



3 Algorithme

3.1 Principe général

Pour chaque individu $\mathbf{x}^{(i)}$ de la base de données :

1. On cherche le neurone **le plus proche** (Best Matching Unit, BMU) sur la carte.
2. On met à jour le BMU et ses voisins pour les rapprocher de $\mathbf{x}^{(i)}$.

3.2 Notations

- $\mathbf{x}^{(i)} \in \mathbb{R}^d$: i -ème individu (vecteur à d variables).
- $\mathbf{w}_j(t) \in \mathbb{R}^d$: vecteur de poids du neurone j à l'itération t .
- $\alpha(t)$: taux d'apprentissage (*learning rate*) à l'itération t .
- $h_{c,j}(t)$: fonction de voisinage entre le BMU c et le neurone j .

3.3 Étape 1 — Initialisation

Les poids $\mathbf{w}_j(0)$ sont initialisés aléatoirement (petites valeurs), ou par échantillonnage aléatoire des données.

3.4 Étape 2 — Recherche du BMU (Best Matching Unit)

Pour un individu $\mathbf{x}^{(i)}$, le BMU est le neurone c dont le vecteur de poids est le plus proche (distance euclidienne) :

$$c = \arg \min_j \|\mathbf{x}^{(i)} - \mathbf{w}_j(t)\|$$

3.5 Étape 3 — Mise à jour des poids

On met à jour le BMU et ses voisins :

$$\mathbf{w}_j(t+1) = \mathbf{w}_j(t) + \alpha(t) \cdot h_{c,j}(t) \cdot (\mathbf{x}^{(i)} - \mathbf{w}_j(t))$$

3.6 Fonction de voisinage

La fonction de voisinage $h_{c,j}(t)$ est souvent une **gaussienne** centrée sur le BMU :

$$h_{c,j}(t) = \exp\left(-\frac{\|\mathbf{r}_c - \mathbf{r}_j\|^2}{2\sigma(t)^2}\right)$$

où $\mathbf{r}_c$ et $\mathbf{r}_j$ sont les positions des neurones c et j sur la grille, et $\sigma(t)$ est le **rayon de voisinage** qui décroît avec le temps.

3.7 Plan de décroissance

Le taux d'apprentissage et le rayon diminuent au fil des itérations :

$$\alpha(t) = \alpha_0 \cdot \exp\left(-\frac{t}{\tau_\alpha}\right), \quad \sigma(t) = \sigma_0 \cdot \exp\left(-\frac{t}{\tau_\sigma}\right)$$

ou linéairement :

$$\alpha(t) = \alpha_0 \left(1 - \frac{t}{T}\right), \quad \sigma(t) = \sigma_0 \left(1 - \frac{t}{T}\right)$$

3.8 Pseudo-code

```

1 def som(X, n_neurons, n_iter, alpha_0, sigma_0):
2     # Initialisation aleatoire des poids
3     W = np.random.randn(n_neurons, d)
4
5     for t in range(n_iter):
6         alpha = alpha_0 * (1 - t / n_iter)
7         sigma = sigma_0 * (1 - t / n_iter)
8
9         # Parcours des donnees
10        for x in X:
11            # 1. BMU

```

```

12     dist = np.linalg.norm(W - x, axis=1)
13     c = np.argmin(dist)
14
15     # 2. Mise à jour
16     for j in range(n_neurons):
17         d_grid = distance_grille(c, j)
18         h = np.exp(-d_grid**2 / (2 * sigma**2))
19         W[j] += alpha * h * (x - W[j])
20
21     return W
    
```

Listing 1 – Pseudo-code SOM

4 Exemple numérique pas à pas

4.1 Données

Considérons $n = 4$ individus en $\mathbb{R}^2$:

$$\mathbf{x}^{(1)} = \begin{pmatrix} 0,2 \\ 0,3 \end{pmatrix}, \mathbf{x}^{(2)} = \begin{pmatrix} 0,1 \\ 0,8 \end{pmatrix}, \mathbf{x}^{(3)} = \begin{pmatrix} 0,7 \\ 0,2 \end{pmatrix}, \mathbf{x}^{(4)} = \begin{pmatrix} 0,9 \\ 0,7 \end{pmatrix}$$

4.2 Architecture

Une carte de 2×2 neurones (4 neurones), organisés en grille carrée.
Positions des neurones sur la grille :

$$\mathbf{r}_0 = (0,0), \mathbf{r}_1 = (0,1), \mathbf{r}_2 = (1,0), \mathbf{r}_3 = (1,1)$$

4.3 Initialisation

Poids initialisés aléatoirement. Pour cet exemple, supposons :

$$\mathbf{w}_0 = \begin{pmatrix} 0,3 \\ 0,8 \end{pmatrix}, \mathbf{w}_1 = \begin{pmatrix} 0,6 \\ 0,4 \end{pmatrix}, \mathbf{w}_2 = \begin{pmatrix} 0,2 \\ 0,1 \end{pmatrix}, \mathbf{w}_3 = \begin{pmatrix} 0,9 \\ 0,5 \end{pmatrix}$$

4.4 Itération 1 — Individu $\mathbf{x}^{(1)} = (0,2; 0,3)$

BMU :

$$\begin{aligned} \|\mathbf{x}^{(1)} - \mathbf{w}_0\| &= \sqrt{(0,2 - 0,3)^2 + (0,3 - 0,8)^2} = \sqrt{0,01 + 0,25} = \sqrt{0,26} \approx 0,510 \\ \|\mathbf{x}^{(1)} - \mathbf{w}_1\| &= \sqrt{(0,2 - 0,6)^2 + (0,3 - 0,4)^2} = \sqrt{0,16 + 0,01} = \sqrt{0,17} \approx 0,412 \\ \|\mathbf{x}^{(1)} - \mathbf{w}_2\| &= \sqrt{(0,2 - 0,2)^2 + (0,3 - 0,1)^2} = \sqrt{0 + 0,04} = \sqrt{0,04} = 0,200 \\ \|\mathbf{x}^{(1)} - \mathbf{w}_3\| &= \sqrt{(0,2 - 0,9)^2 + (0,3 - 0,5)^2} = \sqrt{0,49 + 0,04} = \sqrt{0,53} \approx 0,728 \end{aligned}$$

Le BMU est le neurone $c = 2$ (distance minimale 0,200).

Mise à jour : Prenons $\alpha(0) = 0,5$ et $\sigma(0) = 1,0$.

Distances sur la grille entre $c = 2 = (1,0)$ et chaque neurone :

$$\begin{aligned} d(2,0) &= \|(1,0) - (0,0)\| = 1 & \Rightarrow h_{2,0} &= e^{-1^2/(2 \cdot 1^2)} = e^{-0,5} \approx 0,607 \\ d(2,1) &= \|(1,0) - (0,1)\| = \sqrt{2} \approx 1,414 & \Rightarrow h_{2,1} &= e^{-2/2} = e^{-1} \approx 0,368 \\ d(2,2) &= 0 & \Rightarrow h_{2,2} &= e^0 = 1,000 \\ d(2,3) &= \|(1,0) - (1,1)\| = 1 & \Rightarrow h_{2,3} &= e^{-0,5} \approx 0,607 \end{aligned}$$

Mise à jour des poids ($\Delta \mathbf{w}_j = \alpha \cdot h_{2,j} \cdot (\mathbf{x}^{(1)} - \mathbf{w}_j)$) :

$$\begin{aligned}\mathbf{w}_0 &\leftarrow \mathbf{w}_0 + 0,5 \times 0,607 \times ((0,2; 0,3) - (0,3; 0,8)) \\ &= (0,3; 0,8) + 0,304 \times (-0,1; -0,5) = (0,3; 0,8) + (-0,030; -0,152) = (0,270; 0,648)\end{aligned}$$

$$\begin{aligned}\mathbf{w}_1 &\leftarrow (0,6; 0,4) + 0,5 \times 0,368 \times (-0,4; -0,1) \\ &= (0,6; 0,4) + (-0,074; -0,018) = (0,526; 0,382)\end{aligned}$$

$$\begin{aligned}\mathbf{w}_2 &\leftarrow (0,2; 0,1) + 0,5 \times 1,000 \times (0; 0,2) \\ &= (0,2; 0,1) + (0; 0,100) = (0,200; 0,200)\end{aligned}$$

$$\begin{aligned}\mathbf{w}_3 &\leftarrow (0,9; 0,5) + 0,5 \times 0,607 \times (-0,7; -0,2) \\ &= (0,9; 0,5) + (-0,212; -0,061) = (0,688; 0,439)\end{aligned}$$

4.5 Itération 1 — Individu $\mathbf{x}^{(2)} = (0,1; 0,8)$

BMU :

$$\begin{aligned}\|\mathbf{x}^{(2)} - \mathbf{w}_0\| &= \sqrt{(0,1 - 0,270)^2 + (0,8 - 0,648)^2} = \sqrt{0,029 + 0,023} \approx 0,228 \\ \|\mathbf{x}^{(2)} - \mathbf{w}_1\| &= \sqrt{(0,1 - 0,526)^2 + (0,8 - 0,382)^2} = \sqrt{0,181 + 0,175} \approx 0,597 \\ \|\mathbf{x}^{(2)} - \mathbf{w}_2\| &= \sqrt{(0,1 - 0,200)^2 + (0,8 - 0,200)^2} = \sqrt{0,010 + 0,360} \approx 0,608 \\ \|\mathbf{x}^{(2)} - \mathbf{w}_3\| &= \sqrt{(0,1 - 0,688)^2 + (0,8 - 0,439)^2} = \sqrt{0,346 + 0,130} \approx 0,690\end{aligned}$$

Le BMU est le neurone $c = 0$ (distance 0,228).

Mise à jour :

$$\begin{aligned}\mathbf{w}_0 &\leftarrow (0,270; 0,648) + 0,5 \times 1,000 \times (-0,170; 0,152) = (0,185; 0,724) \\ \mathbf{w}_1 &\leftarrow (0,526; 0,382) + 0,5 \times 0,607 \times (-0,426; 0,418) = (0,397; 0,509) \\ \mathbf{w}_2 &\leftarrow (0,200; 0,200) + 0,5 \times 0,607 \times (-0,100; 0,600) = (0,170; 0,382) \\ \mathbf{w}_3 &\leftarrow (0,688; 0,439) + 0,5 \times 0,368 \times (-0,588; 0,361) = (0,580; 0,505)\end{aligned}$$

4.6 Itération 1 — Individu $\mathbf{x}^{(3)} = (0,7; 0,2)$

BMU : neurone $c = 3$ (vérification laissée en exercice).

Mise à jour (résultats après $\mathbf{x}^{(3)}$) :

$$\mathbf{w}_0 \rightarrow (0,149; 0,706), \mathbf{w}_1 \rightarrow (0,487; 0,335), \mathbf{w}_2 \rightarrow (0,145; 0,350), \mathbf{w}_3 \rightarrow (0,640; 0,416)$$

4.7 Itération 1 — Individu $\mathbf{x}^{(4)} = (0,9; 0,7)$

BMU : neurone $c = 3$.

Mise à jour (poids finaux après 1 passe complète) :

$$\boxed{\mathbf{w}_0 \approx (0,119; 0,677), \mathbf{w}_1 \approx (0,457; 0,403), \mathbf{w}_2 \approx (0,122; 0,335), \mathbf{w}_3 \approx (0,720; 0,487)}$$

4.8 Interprétation

- Le neurone **0** s'est spécialisé sur les points proches de $(0,1; 0,8)$ (individu 2).
- Le neurone **2** attire les points proches de $(0,2; 0,3)$ (individu 1).
- Le neurone **3** attire les points $(0,7; 0,2)$ et $(0,9; 0,7)$ (individus 3 et 4).
- Après plusieurs itérations, la carte s'organise : des neurones voisins représentent des données similaires.

5 Code Python

5.1 Implémentation simple avec NumPy

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 class SOM:
5     def __init__(self, grid_size, d, alpha_0=0.5, sigma_0=1.0):
6         self.grid_size = grid_size # (nx, ny)
7         self.W = np.random.randn(grid_size[0]*grid_size[1], d)
8         self.alpha_0 = alpha_0
9         self.sigma_0 = sigma_0
10        # Positions des neurones sur la grille
11        self.positions = np.array([(i,j) for i in range(grid_size[0])
12                                   for j in range(grid_size[1])])
13
14    def bmu(self, x):
15        dist = np.linalg.norm(self.W - x, axis=1)
16        return np.argmin(dist)
17
18    def fit(self, X, n_epochs=10):
19        n = len(X)
20        for epoch in range(n_epochs):
21            alpha = self.alpha_0 * (1 - epoch/n_epochs)
22            sigma = self.sigma_0 * (1 - epoch/n_epochs)
23            # Melange des donnees a chaque epoque
24            np.random.shuffle(X)
25            for x in X:
26                c = self.bmu(x)
27                for j in range(len(self.W)):
28                    d_grid = np.linalg.norm(
29                        self.positions[c] - self.positions[j])
30                    h = np.exp(-d_grid**2 / (2 * sigma**2))
31                    self.W[j] += alpha * h * (x - self.W[j])
32
33    def predict(self, X):
34        return np.array([self.bmu(x) for x in X])

```

Listing 2 – SOM avec NumPy

5.2 Application sur le jeu de données Iris

```

1 from sklearn.datasets import load_iris
2 import pandas as pd
3
4 iris = load_iris()
5 X = iris.data # 150 individus, 4 variables
6 y = iris.target # 3 especes (pour la visualisation)
7
8 # Normalisation
9 X_norm = (X - X.mean(axis=0)) / X.std(axis=0)
10
11 # Modele SOM (grille 5x5)
12 som = SOM(grid_size=(5,5), d=4)
13 som.fit(X_norm, n_epochs=20)
14
15 # Projection sur la carte
16 clusters = som.predict(X_norm)
17
18 # Visualisation
19 plt.figure(figsize=(8,6))
20 pos = som.positions
21 for i in range(len(X_norm)):
22     plt.scatter(pos[clusters[i],0], pos[clusters[i],1],

```

```

23         c=f'C{y[i]}', s=60, alpha=0.8, edgecolors='k')
24
25 # Grille
26 for j, (px, py) in enumerate(pos):
27     plt.plot(px, py, 'ks', markersize=4, color='gray')
28
29 plt.title("Projection SOM des iris (grille 5x5)")
30 plt.xlabel("Position x sur la grille")
31 plt.ylabel("Position y sur la grille")
32 plt.grid(alpha=0.3)
33 plt.show()

```

Listing 3 – SOM sur Iris

5.3 Avec la bibliothèque MiniSom

```

1 !pip install minisom
2
3 from minisom import MiniSom
4
5 # Normalisation
6 X_norm = (X - X.mean(axis=0)) / X.std(axis=0)
7
8 # Initialisation
9 som = MiniSom(5, 5, 4, sigma=1.0, learning_rate=0.5)
10 som.random_weights_init(X_norm)
11 som.train_random(X_norm, 500)
12
13 # U-Matrix
14 plt.figure(figsize=(8,6))
15 from matplotlib.patches import RegularPolygon
16 from matplotlib.collections import PatchCollection
17
18 # Carte des distances (U-Matrix)
19 U = som.distance_map()
20 plt.imshow(U, cmap='bone_r', origin='upper')
21 plt.colorbar(label='Distance moyenne aux voisins')
22 plt.title("U-Matrix - Carte de Kohonen (Iris)")
23 plt.show()
24
25 # Hit map
26 plt.figure(figsize=(8,6))
27 som_labels = som.labels_map(X_norm, iris.target_names[y])
28 # (affichage des effectifs par cellule - cf. documentation)

```

Listing 4 – SOM avec MiniSom

6 Code R

```

1 # Installation
2 # install.packages("kohonen")
3
4 library(kohonen)
5 library(ggplot2)
6
7 # Jeu de donnees Iris
8 data(iris)
9 X <- as.matrix(iris[,1:4])
10
11 # Normalisation
12 X_norm <- scale(X)
13
14 # Modele SOM (grille hexagonale 5x5)

```

```

15 set.seed(42)
16 grille <- somgrid(xdim=5, ydim=5, topo="hexagonal")
17 som_model <- som(X_norm, grid=grille, rlen=500,
18                 alpha=c(0.5,0.01), radius=c(2,0.1))
19
20 # Visualisation de l'entrainement
21 plot(som_model, type="changes", main="Evolution de la distance")
22
23 # U-Matrix
24 plot(som_model, type="dist.neighbours",
25      main="U-Matrix", palette.name=topo.colors)
26
27 # Clustering
28 plot(som_model, type="codes",
29      main="Vecteurs de poids", palette.name=topo.colors)
30
31 # Carte des effectifs
32 plot(som_model, type="counts",
33      main="Effectifs par neurone")
34
35 # Projection des especes
36 especes <- as.integer(iris$Species)
37 plot(som_model, type="mapping", bg=rainbow(3)[especes],
38      main="Projection des iris", col=rainbow(3)[especes],
39      pch=19, cex=0.8)
40
41 # Composantes
42 plot(som_model, type="property",
43      property=getCodes(som_model,1)[,1],
44      main="Composante 1 (Sepal.Length)",
45      palette.name=topo.colors)

```

Listing 5 – SOM avec le package kohonen

7 Diagnostic et visualisations

7.1 U-Matrix (Unified Distance Matrix)

La U-Matrix représente les **distances moyennes** entre chaque neurone et ses voisins immédiats sur la grille. Une zone claire indique une forte similarité entre neurones voisins (donc un cluster homogène), une zone sombre indique une frontière entre clusters.

7.2 Carte des effectifs (*counts*)

Compte le nombre d'individus projetés sur chaque neurone. Des neurones avec très peu d'individus peuvent être des **outliers** ou des artefacts.

7.3 Plans de composantes (*component planes*)

Pour chaque variable d'origine, on visualise la valeur du poids correspondant sur tous les neurones. Cela permet d'interpréter **quelles variables contribuent à la structure** de la carte.

7.4 Carte de qualité

On peut calculer l'erreur de quantification moyenne (MQE) et l'erreur topologique pour évaluer la qualité de la carte :

$$\text{MQE} = \frac{1}{n} \sum_{i=1}^n \|\mathbf{x}^{(i)} - \mathbf{w}_{c(i)}\|$$

où $c(i)$ est le BMU de $\mathbf{x}^{(i)}$.

8 Résumé — Points clés

Concept	À retenir
Type	Apprentissage non supervisé
Objectif	Réduction de dimension + conservation topologie
BMU	Neurone le plus proche de l'entrée (distance euclidienne)
Mise à jour	$\mathbf{w}_j \leftarrow \mathbf{w}_j + \alpha h_{c,j} (\mathbf{x} - \mathbf{w}_j)$
Voisinage	Gaussien : $h_{c,j} = \exp(-\ \mathbf{r}_c - \mathbf{r}_j\ ^2 / 2\sigma^2)$
Décroissance	α et σ diminuent avec le temps
Visualisations	U-Matrix, composantes, effectifs, projection

QCM — Vérifiez vos connaissances

Pour chaque question, choisissez la ou les bonnes réponses. Les réponses sont en page 11.

- Que signifie l'acronyme **SOM** ?
 - Self-Organizing Machine
 - Self-Organizing Map
 - Systematic Ordered Mapping
 - Supervised Objective Method
- Les cartes de Kohonen sont une méthode d'apprentissage :
 - supervisé
 - non supervisé
 - par renforcement
 - semi-supervisé
- Comment détermine-t-on le **BMU** (Best Matching Unit) pour un vecteur d'entrée $\mathbf{x}$?
 - En maximisant la distance à $\mathbf{x}$
 - En minimisant la distance euclidienne à $\mathbf{x}$
 - En calculant la moyenne de tous les poids
 - En choisissant aléatoirement un neurone
- La fonction de voisinage $h_{c,j}$ utilisée dans la mise à jour des poids est typiquement :
 - une fonction linéaire
 - une fonction exponentielle décroissante
 - une gaussienne
 - une fonction sinus
- Quel est le rôle du **taux d'apprentissage** $\alpha(t)$ dans l'algorithme SOM ?
 - Il détermine la taille de la grille
 - Il contrôle l'ampleur de la mise à jour des poids
 - Il fixe le nombre d'itérations
 - Il initialise les poids
- Au fil des itérations, le taux d'apprentissage α et le rayon de voisinage σ :
 - augmentent
 - restent constants
 - diminuent
 - oscillent
- Que visualise la **U-Matrix** (Unified Distance Matrix) ?

- (a) Les valeurs des poids de chaque neurone
 - (b) Les distances moyennes entre chaque neurone et ses voisins
 - (c) Le nombre d'individus par neurone
 - (d) L'évolution de l'erreur au cours de l'apprentissage
8. En Python, quelle bibliothèque légère est dédiée aux cartes de Kohonen ?
- (a) scikit-learn
 - (b) MiniSom
 - (c) TensorFlow
 - (d) Keras
9. En R, quel package permet d'implémenter les cartes de Kohonen ?
- (a) ggplot2
 - (b) kohonen
 - (c) randomForest
 - (d) caret
10. Quelle est la **propriété fondamentale** des cartes de Kohonen ?
- (a) Elles maximisent la variance inter-classe
 - (b) Elles préservent la topologie des données d'entrée
 - (c) Elles minimisent l'erreur de prédiction
 - (d) Elles nécessitent des données étiquetées

Réponses au QCM

N°	Explication	Ré
1	SOM = Self-Organizing Map (carte auto-organisatrice).	b
2	Les cartes de Kohonen sont une méthode d'apprentissage non supervisé : on ne fournit pas les étiquettes.	b
3	Le BMU est le neurone dont le vecteur de poids est le plus proche de $\mathbf{x}$ au sens de la distance euclidienne.	b
4	Le voisinage gaussien $h_{c,j} = \exp(-\ \mathbf{r}_c - \mathbf{r}_j\ ^2/2\sigma^2)$ est le plus courant.	c
5	α contrôle la vitesse d'apprentissage : plus α est grand, plus les poids sont modifiés.	b
6	α et σ diminuent avec le temps pour permettre une convergence stable.	c
7	La U-Matrix représente les distances entre neurones voisins : zones claires = clusters.	b
8	MiniSom est une bibliothèque Python légère dédiée aux cartes de Kohonen.	b
9	Le package <code>kohonen</code> est le plus utilisé en R pour les SOM.	b
10	Les cartes de Kohonen préservent la topologie : des points proches en entrée restent proches sur la carte.	b

Fin de la fiche.